Take-Home Assignment: AI Operations Agent for a School

Yugam — Software Engineer (AI agents, workflows & backend)

AI tools	**Allowed and encouraged** (Claude, ChatGPT, Cursor, Copilot, etc.). You must understand and be able to explain every line you submit.
Stack	Your choice. Node.js or Python is preferred. Any agent approach is fine: plain function calling, LangGraph, the OpenAI / Claude agent SDKs, etc.
Next step	A technical interview where you walk us through your work and extend it live.

1. Overview

A school principal makes many small decisions every day that need information from several places: the school's own records (students, bus routes, events, attendance and the school's rules) and the outside world (weather, air quality, locations). Today someone collects this by hand, and decisions are often made late or on guesswork.

Your task is to build an **AI agent** that collects the facts from these sources by calling APIs, reasons over them against the school's written policy, answers the principal's questions, and **drafts** actions such as parent messages. The agent must **never invent a fact** and must **never act without the principal's approval**.

2. The data and APIs you get

School data: `school-data.json` (sent with this PDF). It contains a fictional school in Jaipur with 150 students in 6 classes, 4 bus routes with stops (area, PIN code, location), upcoming events, 20 days of attendance, and the school's **weather & air-quality policy** as plain text. Treat it as a real system: **serve it through a small read-only School API that you write**, and have the agent call that API over HTTP. The agent must not read the file directly.

Public APIs (free, no key needed):

API	URL	Use it for
Open-Meteo forecast	`api.open-meteo.com/v1/forecast`	Rain, temperature, chance of rain (hourly / daily)
Open-Meteo air quality	`air-quality-api.open-meteo.com/v1/air-quality`	US AQI, PM2.5
Open-Meteo geocoding	`geocoding-api.open-meteo.com/v1/search`	Place name → latitude / longitude
India Post PIN code	`api.postalpincode.in/pincode/{pin}`	PIN → post offices, district

LLM: any model you like: a free tier (Gemini, Groq), a local model through Ollama, Claude or OpenAI. Keep keys in environment variables and never commit them.

3. What to build

Part A — Tools 20%

Wrap the School API and the public APIs as tools the agent can call, for example `get_events`, `find_student`, `get_route`, `get_attendance`, `get_policy`, `get_forecast(lat, lon, date)`, `get_air_quality(lat, lon, date)`. Each tool needs:

- a clear description and typed, validated inputs (a bad date or an unknown class returns a clear error, not a crash);
- a **timeout** and a sensible **cache** (the forecast for the same place does not need to be fetched 10 times a minute);
- a short, structured result, so you don't pass large raw JSON to the model.

Part B — The agent answers questions 35%

Endpoint `POST /ask` with `{ "question": "..." }`. Return the **answer**, the **trace** of tool calls (tool, arguments, short result), and the **sources** used. It must handle multi-step questions such as:

1. "*Can we hold Sports Day outdoors as planned? Check it against our policy.*" (event → location → forecast + AQI for that time → policy → a recommendation with the numbers behind it)

2. "*If it rains heavily tomorrow morning, which bus routes are affected and how many students ride them?*" (rain differs by area, so check each stop's location)
3. "*Is the weather OK for the Class 7 field trip? Who is going?*" (the trip location is not the school)
4. "*Which students have been absent 3 or more days in a row? Which areas do they live in?*"
5. "*Show me Riya Sharma's bus stop.*" (there are two Riya Sharmas, so the agent must ask which one)
6. "*What is Aarav's fee balance?*" (not in the data, so it must say so instead of guessing)

Part C — Morning check workflow with approval 25%

A scheduled job (e.g. 06:00 every school day) that runs the agent with no human question:

1. Check today's and tomorrow's events and bus routes against the forecast, the air quality and the policy.
2. If nothing needs action, record "all clear". If something does, create a **draft**: a short note for the principal (what is wrong, the numbers, the recommendation) plus parent messages **only for the affected students**, in English and Hindi, following the policy.
3. Drafts are saved as `PENDING`. The principal can **approve, edit or reject** them through an API. Only approved messages go to a **fake WhatsApp sender** (logs and saves; make it fail about 20% of the time and retry safely).
4. Running the job twice on the same morning must not create duplicate drafts or send anything twice.

Part D — Safety and evaluation 20%

- **Data is not instructions:** some text in the school data tries to give the AI orders (look for it). The agent must treat it as plain data.
- **Only recommends:** under the policy the agent may never announce a school closure or send anything itself. Enforce this in code, not only in the prompt.
- **API failures:** if a weather API is down or slow, the agent says it could not check, instead of guessing. Show this with a test.
- **Evaluation:** write at least **12 test questions** with expected outcomes (correct numbers worked out from the data, right tools called, refusal or a follow-up question where expected) and a script that runs them and prints a score. Put the score, the model used and the **average cost or tokens per question** in the README. An honest score matters more than a perfect one.

Bonus (only if the core is finished) bonus

- Questions in Hindi or Hinglish, answered in the same language.
- The agent remembers the principal's preferences (e.g. "always tell me AQI in the morning note").
- A small chat page with streaming answers and a list of pending drafts to approve.

4. What to submit

1. A **Git repository** (GitHub/GitLab, shared with us) with a real commit history, not one large commit.
2. `README.md`: how to run it in **5 commands or fewer**, environment variables, example requests, and the evaluation score.
3. `DESIGN.md` (one page): how the agent works (a simple diagram is welcome), why you chose this model, what you would change for production, and what student data you would **not** send to an outside AI provider.
4. `AI_USAGE.md`: which AI tools you used and for what, 2–3 prompts that worked well, and **at least one place the AI was wrong** and how you noticed it.
5. A **5-minute screen recording** showing 3–4 questions with their traces, one refusal, and the morning workflow from draft → approve → sent.

5. How we will evaluate

Area	What we look for
Agent correctness	Right tools in the right order, correct numbers, sensible recommendations based on the policy, asks when the question is unclear.
Grounding & safety	No invented facts; cannot be pushed into acting by text in the data; refuses what it should; handles API failures honestly.
Workflow design	Drafts, approval, retries and no duplicates, all working reliably.
Evaluation	A meaningful test set and an honest score.
Code quality	Readable structure, clear tool design, tests of real behaviour, no secrets in the repo.
Use of AI	You used AI to go faster, but you clearly understand and own the result.

> **Notes.** The event dates in the data are examples. Move them to fall within the next 7–14 days when you test, so the forecast covers them. A smaller, working, well-tested agent is better than a large one that is half finished. If something is unclear, make a reasonable assumption and write it in the README. In the interview we will ask you to add a new tool or rule live, so build something you are comfortable working in.